Local Code RAG Workshop — Windows Cheat Sheet

Nov 7, 2026 · @Boris Wrubel

Only what differs on Windows — everything else follows the workshop guides in `docs/`. Not yet tested end to end on Windows.

Install

Same components as on Linux, but with Windows installers and two path rules.

Item	On Windows
Python	installer from python.org, tick “Add Python to PATH”; avoid the Microsoft Store version; call it with `py`
goose CLI	in PowerShell: `Invoke-WebRequest -Uri "https://raw.githubusercontent.com/block/goose/main/download_cli.ps1" -OutFile "download_cli.ps1"`, then `.\download_cli.ps1`; add `%USERPROFILE%\.local\bin` to PATH
goose desktop	check the GitHub releases for a Windows build
Ollama	installer from ollama.com; runs in the tray, not as a service
Repos	clone into a path **without spaces**, e.g. `C:\work` — not `C:\Users\Max Mustermann\…`
Terminal	Windows Terminal with PowerShell

Settings

Ollama takes its settings from user environment variables instead of a systemd override (Ollama FAQ):

1. Quit Ollama in the tray
2. Settings → search “environment variables” → *Edit environment variables for your account*

3. Add `OLLAMA_CONTEXT_LENGTH` = `32768` (16 GB RAM: `16384`); with an integrated GPU also `OLLAMA_IGPU_ENABLE` = `1`
4. OK, then start Ollama from the Start menu

The goose config lives in `%APPDATA%\Block\goose\config\config.yaml` instead of `~/.config/goose/config.yaml` (config files). Add `GOOSE_CONTEXT_LIMIT: 32768` there and check for `active_provider: ollama`.

Commands

The guides use Linux syntax; translate it with these rules.

In the guides	On Windows (PowerShell)
`python3 index.py …`	`py index.py …`
`python3 -m venv .venv`	`py -m venv .venv`
`source .venv/bin/activate`	`.venv\Scripts\Activate.ps1`
`../generated_kotlin`	`..\generated_kotlin`
`./scripts/check_setup.py`	`py scripts\check_setup.py`
`EMBED_MODEL=qwen3-embedding python3 index.py …`	`$env:EMBED_MODEL="qwen3-embedding"; py index.py …` — stays set until the terminal closes, `Remove-Item Env:EMBED_MODEL` removes it
`mvn test -Dcucumber.filter.tags=@search`	`mvn test "-Dcucumber.filter.tags=@search"` — quote every `-D…` argument
`cmd1 && cmd2`	`cmd1; cmd2`
`sudo systemctl …` (Ollama)	see Settings
`curl … | sh` (installers)	see Install

goose extension

The extension command uses the venv's `python.exe` under `Scripts`, not `bin/python`:

```
C:\work\local-code-rag-workshop\.venv\Scripts\python.exe C:\work\local-code-rag-workshop\mcp_server.py
```

`py scripts\goose_extension.py` prints it for your folder. Paste it into a terminal first: `[local-code-rag] starting …` and then silence means it works (Ctrl+C to stop). The config file it mentions is the Linux path — on Windows use the one under Settings.

Gotchas

Two checks in `check_setup.py` fail on Windows even when the setup is fine — check those by hand.

Symptom	Cause	Fix
Setup check: "goose not found"	the script looks for `goose`, not `goose.exe`	check by hand: `goose --version`
Setup check: goose config not found	the script looks in `~/.config/goose`	open the config under `%APPDATA%`, look for `active_provider: ollama`
`Activate.ps1 cannot be loaded`	PowerShell execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`python3` opens the Microsoft Store	Store alias instead of python.org Python	use `py`
`goose` not recognised after install	`%USERPROFILE%\.local\bin` not in PATH	add it, open a new terminal
Maven: `Unknown lifecycle phase ".execution.dry-run=true"`	PowerShell splits `-D` arguments at the dot	quote every `-D…` argument
Extension fails to start	path with spaces	move the repo, e.g. to `C:\work`
Ollama ignores the new variable	Ollama was still running	quit it in the tray, start it again
Garbled emoji or colour codes	old console	use Windows Terminal
Sources shown with `\`	paths stored as on disk	expected

Sources: goose install guide · goose config files · Ollama FAQ